

A. Setup (5–10 min)

1. Create a project folder

```
de-section1-lab/  
├── data/bronze  
├── data/silver  
├── data/gold  
└── lab1.py
```

2. Create & activate a virtual env (pick one):

- **macOS/Linux**

```
python3 -m venv .venv  
source .venv/bin/activate
```

- **Windows (PowerShell)**

```
python -m venv .venv  
.venv\Scripts\Activate.ps1
```

3. Install packages

```
pip install -U pip  
pip install pandas pyarrow duckdb pandera
```

(Pandera is a lightweight validator we'll use for quick quality checks.)

Prefer notebooks? Do this in Google Colab; skip the venv and just pip install the same libraries in a cell.

B. Ingest → Bronze (10 min)

Use a small, public dataset so everyone can run it easily. We'll use **seaborn “tips”** (restaurant bills).

[lab1.py](#) (start this file):

```
from pathlib import Path
```

```
import pandas as pd
```

```
BRONZE = Path("data/bronze")
```

```
SILVER = Path("data/silver")
```

```
GOLD = Path("data/gold")
```

```
for p in [BRONZE, SILVER, GOLD]:
```

```
p.mkdir(parents=True, exist_ok=True)
```

```
RAW_URL = "https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv"
```

```
def extract_to_bronze():
```

```
df = pd.read_csv(RAW_URL)
```

```
out_path = BRONZE / "tips_raw.csv"
```

```
df.to_csv(out_path, index=False)
```

```
print(f"[BRONZE] wrote {out_path} with {len(df)} rows")
```

```
return out_path
```

```
if name == "main":
```

```
extract_to_bronze()
```

Run:

```
python lab1.py
```

C. Clean/Transform → Silver (15–20 min)

- Standardize column names
- Derive **tip_pct**
- Add a synthetic **visit_datetime** (so we can demo time-based flows)
- Write **Parquet** (columnar, compressed)

Append to [lab1.py](#):

```
import numpy as np
```

```
def transform_to_silver(bronze_path: Path):
```

```
df = pd.read_csv(bronze_path)
```

```

# standardize columns
df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]

# derived columns
df["tip_pct"] = (df["tip"] / df["total_bill"]).round(4)

# simple synthetic timestamp for demo (hourly sequence)
df["visit_datetime"] = pd.date_range("2024-01-01", periods=len(df), freq="H")

# basic type tweaks
cat_cols = ["sex", "smoker", "day", "time"]
for c in cat_cols:
    df[c] = df[c].astype("category")

out_path = SILVER / "tips_clean.parquet"
df.to_parquet(out_path, index=False) # needs pyarrow
print(f"[SILVER] wrote {out_path} with {len(df)} rows")
return out_path

if name == "main":
    p = extract_to_bronze()
    transform_to_silver$

```

Run:

python [lab1.py](#)

D. Aggregate/Serve → Gold (10 min)

Create a tidy analytics table.

Append:

```

def aggregate_to_gold(silver_path: Path):
    df = pd.read_parquet(silver_path)
    summary = (
        df.groupby(["sex", "smoker", "day"], as_index=False)
        .agg(

```

```

avg_tip_pct = ("tip_pct", "mean"),
total_revenue = ("total_bill", "sum"),
rows = ("tip_pct", "count")
)
)
out_path = GOLD / "tips_summary.parquet"
summary.to_parquet(out_path, index=False)
print(f"[GOLD] wrote {out_path} with {len(summary)} rows")
return out_path

if name == "main":
    p = extract_to_bronze()
    s = transform_to_silver$
    aggregate_to_gold(s)

```

E. Query Gold/Silver with SQL (DuckDB) (10 min)

Show the “data warehouse” feel without setting one up.

Append:

```

import duckdb

def query_examples():
    con = duckdb.connect(database=":memory:")
    # Query silver directly from Parquet
    q1 = con.execute("""
SELECT sex, smoker, AVG(tip_pct) AS avg_tip_pct, SUM(total_bill) AS revenue
FROM read_parquet('data/silver/tips_clean.parquet')
GROUP BY 1,2
ORDER BY avg_tip_pct DESC
""").df()
    print("\n[SQL on Silver]\n", q1.head())

    # Query gold
    q2 = con.execute("""

```

```
SELECT day, AVG(avg_tip_pct) AS avg_tip_pct
FROM read_parquet('data/gold/tips_summary.parquet')
GROUP BY day
ORDER BY avg_tip_pct DESC
""").df()
print("\n[SQL on Gold]\n", q2)
```

```
if name == "main":
    p = extract_to_bronze()
    s = transform_to_silver$
    aggregate_to_gold(s)
    query_examples()
```

Run:

```
python lab1.py
```

F. Basic Data Quality Checks (Pandera) (10 min)

Validate common rules before promoting to silver/gold.

Append:

```
import pandera as pa
from pandera import Column, Check

def validate_silver(silver_path: Path):
    schema = pa.DataFrameSchema({
        "total_bill": Column(float, Check.ge(0)),
        "tip": Column(float, Check.ge(0)),
        "tip_pct": Column(float, Check.in_range(0, 1), nullable=False),
        "size": Column(int, Check.ge(1)),
        "visit_datetime": Column(pa.DateTime),
    }, coerce=True)

    df = pd.read_parquet(silver_path)
    schema.validate(df, lazy=True) # raises if violations
```

```
print("[QUALITY] Silver passed validation!")
```

```
if name == "main":  
    p = extract_to_bronze()  
    s = transform_to_silver$  
    validate_silver(s)  
    aggregate_to_gold(s)  
    query_examples()
```

Talk track for Lecture 3: you just touched **file formats (CSV→Parquet)**, **pandas ETL**, **SQL-on-lake with DuckDB**, and **data validation**. These are foundational tools in a modern data stack.

G. (Optional) Schedule It (10–20 min)

Option 1 — Cron (simple & universal)

- **macOS/Linux:** crontab -e and add:

```
0 8 * * * cd /full/path/to/de-section1-lab && .venv/bin/python lab1.py >> run.log 2>&1
```

- **Windows:** Use **Task Scheduler** → *Create Basic Task* → trigger *Daily 8:00 AM* → action *Start a Program* → python.exe with Arguments: C:\path\to\[lab1.py](#). Set *Start in* to your project folder.

(You'll cover Airflow/Prefect properly in later sections; this gives students a quick win now.)

H. What to Submit (for grading)

1. Screenshot of your project tree showing **bronze/silver/gold** Parquet files.
 2. Console output showing **[QUALITY] Silver passed validation!** and SQL results.
 3. A short README (5–8 bullets) explaining:
 - Source URL, transformations, and new columns
 - Why Parquet for silver/gold
 - One insight from your SQL query
-

I. Stretch Goals (optional)

- Swap dataset for a bigger one (e.g., NYC taxi; keep nrows=50000 on ingest).
 - Partition silver/gold by **day** (partition_cols=["day"]) when writing Parquet.
 - Add a second gold table (e.g., **avg_tip_pct by time**).
 - Replace Pandera with **Great Expectations** or add a few GE checks.
 - Containerize with a minimal **Dockerfile**.
-

Recap: Mapping to the Lectures

- **Lecture 1 (What is DE?):** You built a real ETL pipeline.
- **Lecture 2 (Data Ecosystem):** You organized layers (bronze/silver/gold) and used columnar storage (Parquet).
- **Lecture 3 (Tools & Tech):** You used pandas/pyarrow, DuckDB (SQL-on-lake), and Pandera for quality; learned basic scheduling.